

CS2023 - Programación III
Programación concurrente
Ejercicios de programación concurrente
Rubén Rivas Medina
Mayo 2025

Implementa las nueve preguntas en C++20. Coloca cada declaración en el archivo de `include/` indicado y la implementación correspondiente en `src/`. Los ejemplos usan rutas temporales o nombres de archivo ilustrativos; las funciones deben respetar los parámetros recibidos. Espera a que terminen los hilos antes de devolver un resultado. Cada archivo de entrada contiene enteros separados por espacios o saltos de línea.

1. Uso de hilos con archivos (`std::thread`)

Ejercicio 1: clasificación de números

Interfaz — `include/clasificacion_numeros.h`:

```
void clasificar_numeros(const std::string& entrada = "numeros.txt",
                        const std::string& directorio_salida = ".");
```

Lee los enteros de `entrada` y crea cuatro hilos, uno por archivo de salida: `pares.txt`, `mult3.txt`, `mult5.txt` y `otros.txt`. Un número divisible por varios divisores debe aparecer en cada archivo correspondiente. `otros.txt` contiene solo los que no son divisibles por 2, 3 ni 5. Conserva el orden de entrada dentro de cada archivo; crea los cuatro archivos incluso si la entrada está vacía.
Ejemplo de uso. Si `numeros.txt` contiene `2 3 5 7 30`:

```
clasificar_numeros("numeros.txt", "salida");
// salida/pares.txt -> 2 30
// salida/mult3.txt -> 3 30
// salida/mult5.txt -> 5 30
// salida/otros.txt -> 7
```

Casos de prueba:
1. Entrada `2 3 5 7 30`: comprobar las cuatro secuencias anteriores.
2. Entrada vacía: comprobar que existen cuatro archivos vacíos.
3. Entrada `6 10 15`: comprobar que `6` aparece en pares y mult3, `10` en pares y mult5, y `15` en mult3 y mult5.

Ejercicio 2: suma por rangos

Interfaz — `include/suma_rangos.h`:

```
std::array<long long, 10> sumar_rangos(
    const std::string& archivo = "numeros.txt");
```

Lanza diez hilos para sumar los valores de los intervalos inclusivos 1–10, 11–20, …, 91–100. La posición `i` del arreglo corresponde al intervalo `10*i+1` a `10*i+10`. Conserva las repeticiones; ignora los valores fuera de 1–100. Un intervalo sin valores suma cero.
Ejemplo de uso. Si `numeros.txt` contiene `1 10 11 20 0 101`:

```
auto sumas = sumar_rangos("numeros.txt");
// Resultado: {11, 31, 0, 0, 0, 0, 0, 0, 0, 0}
```

Casos de prueba:
1. Entrada `1 10 11 20`: posiciones 0 y 1 iguales a 11 y 31.
2. Entrada `1 1 10 0 101`: posición 0 igual a 12; las demás, cero.
3. Entrada vacía: diez ceros.

Ejercicio 3: suma de archivos clasificados

Interfaz — `include/suma_archivos.h`:

```
std::array<long long, 4> sumar_archivos_clasificados(
    const std::string& directorio = ".");
```

Lee `pares.txt`, `mult3.txt`, `mult5.txt` y `otros.txt` del directorio indicado. Usa cuatro hilos para calcular una suma por archivo y devuelve las sumas en ese orden. Si falta un archivo, informa el error mediante una excepción.
Ejemplo de uso. En `salida/`, los cuatro archivos contienen `2 4 6`, `3 6 9`, `5 10` y `7 11`:

```
auto sumas = sumar_archivos_clasificados("salida");
// Resultado: {12, 18, 15, 18}
// Orden: pares, mult3, mult5, otros
```

Casos de prueba:
1. Los cuatro archivos del ejemplo: resultado `{12, 18, 15, 18}`.
2. Cuatro archivos vacíos: resultado `{0, 0, 0, 0}`.
3. Falta `mult5.txt`: la función lanza una excepción.

2. Protección con mutex (`std::mutex`)

Ejercicio 4: frecuencias

Interfaz — `include/frecuencia_numeros.h`:

```
std::unordered_map<int, int> contar_frecuencias(
    const std::string& archivo = "numeros.txt",
    std::size_t numero_hilos = 4);
```

Reparte los enteros del archivo entre los hilos y cuenta sus apariciones en un mapa compartido. Protege las actualizaciones del mapa con `std::mutex`. Acepta un archivo vacío y una cantidad de hilos mayor que la de datos. `numero_hilos` debe ser positivo.
Ejemplo de uso. Si `numeros.txt` contiene `1 2 1 3 2 1`:

```
auto frecuencias = contar_frecuencias("numeros.txt", 3);
// frecuencias.at(1) == 3
// frecuencias.at(2) == 2
// frecuencias.at(3) == 1
```

Casos de prueba:
1. Entrada `1 2 1 3 2 1` y 3 hilos: frecuencias 3, 2 y 1.
2. Entrada vacía y 4 hilos: mapa vacío.
3. Entrada `9 9` y 10 hilos: frecuencia de 9 igual a 2.

Ejercicio 5: clientes bancarios

Interfaz — `include/cliente_bancario.h`:

```
class Client {
public:
    Client();
    explicit Client(double saldo_inicial);
    void deposit(double cantidad);
    void withdraw(double cantidad);
    double getBalance() const;
    friend std::ostream& operator<<(std::ostream&, const Client&);
    // Agrega los atributos privados necesarios.
};
void simular_operaciones(Client& cliente, int nd, double d,
                         int nr, double r);
std::vector<double> simular_clientes(int m, double saldo_inicial,
                                     int nd, double d, int nr, double r);
```

El saldo inicial del constructor sin argumentos es cero. Protege el saldo con un mutex en depósitos, retiros y lecturas. `simular_operaciones` ejecuta `nd` depósitos de `d` y `nr` retiros de `r` concurrentemente. `simular_clientes` crea `m` clientes independientes, ejecuta esa simulación para cada uno y devuelve sus saldos en orden. El retiro resta la cantidad incluso si el saldo pasa a ser negativo. Rechaza cantidades y conteos negativos con una excepción.

Ejemplo de uso:

```
Client cliente(100);
simular_operaciones(cliente, 5, 3, 5, 2);
std::cout << cliente;  // Imprime: 105
auto saldos = simular_clientes(2, 100, 5, 3, 5, 2);
// Resultado: {105, 105}
```

Casos de prueba:

1. `Client{}`: saldo cero; `Client{100}` tras 5 depósitos de 3 y 5 retiros de 2: saldo 105.
2. Diez hilos realizan cinco depósitos de 3 y cinco retiros de 2 cada uno sobre un saldo de 1000: saldo final 1050.
3. Dos clientes con saldo inicial 100 y la simulación del ejemplo: saldos `{105, 105}`.

Ejercicio 6: registro compartido

Interfaz — `include/logger.h`:

```
class Logger {
public:
    explicit Logger(std::string archivo = "log.txt");
    void log(std::string mensaje);
    // Agrega los atributos privados necesarios.
};
void clasificar_y_registrar(
    Logger& logger, const std::string& entrada = "numeros.txt",
    const std::string& directorio_salida = ".");
```

`Logger::log` agrega el mensaje y un salto de línea al archivo, en modo **append**. Un mutex interno debe proteger la escritura completa. `clasificar_y_registrar` realiza la clasificación del ejercicio 1 con cuatro hilos y registra una línea por archivo con el formato `nombre.txt: cantidad`. El orden de las cuatro líneas del registro puede variar.
Ejemplo de uso. Si `numeros.txt` contiene `2 3 5 7 30`:

```
Logger logger("log.txt");
clasificar_y_registrar(logger, "numeros.txt", "salida");
// log.txt contiene (en cualquier orden):
// pares.txt: 2
// mult3.txt: 2
// mult5.txt: 2
// otros.txt: 1
```

Casos de prueba:
1. Dos llamadas `log("uno")` y `log("dos")`: el archivo contiene `uno` y `dos` en líneas consecutivas.
2. Archivo con una línea previa `anterior`: una llamada a `log("nuevo")` conserva ambas líneas.
3. Clasificación del ejemplo: cuatro líneas con las cantidades indicadas, sin duplicados ni líneas mezcladas.

3. Coordinación con semáforo binario (`std::binary_semaphore`)

Ejercicio 7: productor y consumidor

Interfaz — `include/productor_consumidor.h`:

```
std::vector<int> productor_consumidor(
    std::size_t cantidad, unsigned semilla, std::ostream& salida);
```

Implementa un productor que genera `cantidad` enteros entre 1 y 1000 con la semilla dada y un consumidor que toma cada valor de un buffer de un solo elemento. Coordina ambos hilos con semáforos binarios para impedir sobrescrituras o lecturas antes de producir. El consumidor escribe un entero por línea en `salida`; la función devuelve esos mismos enteros, en el mismo orden. Una misma semilla debe producir la misma secuencia.
Ejemplo de uso:

```
std::ostringstream salida;
auto valores = productor_consumidor(3, 42, salida);
// valores.size() == 3
// salida.str(): esos mismos 3 enteros, uno por línea.
```

Casos de prueba:
1. `cantidad = 0`: vector y salida vacíos.
2. `cantidad = 10`: diez valores entre 1 y 1000; los valores impresos coinciden con el vector.
3. Dos llamadas con `cantidad = 100` y `semilla = 42`: vectores y salidas iguales.

Ejercicio 8: lectura cíclica

Interfaz — `include/lectura_ciclica.h`:

```
void leer_archivos_en_turnos(
    const std::array<std::string, 3>& archivos,
    std::ostream& salida);
```

Crea tres hilos asociados, respectivamente, con `A.txt`, `B.txt` y `C.txt`. Usa semáforos binarios para imprimir un entero por línea en turnos A→B→C→A. Si un archivo se agota, omite sus turnos y continúa con los que aún contienen datos. Termina cuando los tres se agotan.

Ejemplo de uso. Los archivos contienen `A.txt: 1 4 7 10`, `B.txt: 2 5 8` y `C.txt: 3 6 9`:

```
leer_archivos_en_turnos({"A.txt", "B.txt", "C.txt"}, std::cout);
// Imprime: 1 2 3 4 5 6 7 8 9 10
// Un número por línea.
```

Casos de prueba:

1. Los tres archivos del ejemplo: salida `1 2 3 4 5 6 7 8 9 10`.
2. `A.txt: 1 3`, `B.txt` vacío y `C.txt: 2`: salida `1 2 3`.
3. Tres archivos vacíos: salida vacía y terminación normal.

Ejercicio 9: acceso exclusivo a una sala

Interfaz — `include/sala.h`:

```
void ingresar_a_la_sala(
    std::string nombre, std::ostream& salida = std::cout,
    std::chrono::milliseconds permanencia =
        std::chrono::milliseconds(1));
```

Usa un semáforo binario compartido para permitir una sola persona en la sala a la vez. Cada llamada escribe `nombre ha entrado a la sala`, espera `permanencia`, escribe `nombre ha salido de la sala` y libera el acceso. En llamadas concurrentes, el orden entre personas puede variar, pero cada mensaje de entrada debe ir inmediatamente seguido del mensaje de salida de esa misma persona.

Ejemplo de uso:

```
std::ostringstream salida;
std::thread a([&] { ingresar_a_la_sala("Ana", salida); });
std::thread b([&] { ingresar_a_la_sala("Luis", salida); });
a.join();
b.join();
// Dos pares completos: entrada/salida de Ana y de Luis.
// Cualquiera de los pares puede aparecer primero.
```

Casos de prueba:

1. Una llamada para `Ana`: exactamente las líneas de entrada y salida de Ana, en ese orden.
2. Cinco hilos con nombres diferentes: diez líneas, cada entrada seguida por su salida correspondiente.
3. Cien visitas concurrentes con permanencia de cero milisegundos: doscientas líneas en pares completos y ningún nombre perdido.